

seL4 as ARM TrustZone SWd OS

- How to setup a communication protocol between NWd apps and SWd apps?
 - How to handle Interrupts
 - How to process SMC and interact with ATF/NWD?
-

Work in progress

- just writing down a few thoughts about how things could work.
- should draw a picture of the state transitions...
- this requires seL4 kernel modifications - assess consequences and how much this breaks the current verification

How to setup a communication protocol between NWd apps and SWd apps?

- at a high lever, there should be a virtual network card that connect NWd to the SOES in TZ via sockets
 - having SEOS TZ services accessible like "internet servers" sounds nice
 - maybe we don't need full IP sockets here, but Unix sockets also work
- need a lower layer that implement the actual communication
 - see if we can use the method from the OP-TEE driver here

How to handle Interrupts

- configure SWd interrupts to trap as IRQ when executing in SWd (ie "internal interrpts")
 - no change to the standard seL4 behavior
 - code flow
 - enter at vector table in src/arch/arm/64/traps.S
 - cur_el_irq if interrupting the idle thread
 - lower_el_irq if interrupting a thread, saves current thread state in stack
 - calls c_handle_interrupt() in src/arch/arm/c_traps.c with all regs available
 - could pass IRQ/FIQ entry flag here
 - calls handleInterruptEntry() from src/api/syscall.c
- configure non-SWd interrupts to trap as FIQ when executing in SWd (ie "external interrpts")
 - Xilinx Ultrascale specific
 - see TRM, chapter "APU Interrupt Grouping and Virtualization"
 - unfortunately, platform has GICv2
 - 2 groups for secure and non-secure interrupts
 - can't separate ATF and SWd interrupt as both are "secure".
 - SWd must explicitly know all interrupts used by ATF
 - if we are lucky, there there are none or just a few fixed ATF interrupts
 - GICv3 would support 3 groups, so SWd would not need to know details
 - see <https://community.arm.com/developer/ip-products/processors/b/processors-ip-blog/posts/programmable-interrupt-controllers-a-new-architecture>
 - notify TZ-Thread of pending interrupt
 - TZ-Thread will not ACK the interrupt, but simply use SMC to lave SWd
 - could simply be S-YIELD, then interrupt would trap in ATF
 - or define a dedicated SMC command to notify ATF we left due to an interrupt
 - next entry to SWd will simply be in TZ-Thread after the SCM instruction.
 - since we can enter SWd only vie SMC from ATF, TZ-Thread looks at SMC parameters
 - could be a command
 - or S-YIELD
 - but also we remember that we left SWd due to interrupt, so SWd is likely still busy
 - it it's a command, then dispatch command, otherwise just continue as if interrupt was magically handled

How to process SMC and interact with ATF/NWD?

- define a "TZ Thread", which is a seL4 "kernel thread" like the idle thread.
 - must be a kernel thread, as only kernel mode can use SMC instruction
- "TZ Thread" has an endpoint that can receive IPCs
 - sending an IPCs to the TZ-Thread allows others to triggering SMCs
 - can manage access to SMCs with existing seL4 capabilities model
 - only kernel and "TZ Server" thread running in user mode should be allowed to do IPC with TZ-Thread
 - TZ Server can implement all the SMC command access guarding in user mode
- if TZ-Thread executes a SMC command, then SWd is left
 - we assume SWd only issues SMCs for three reasons
 - synchronous ATF Command to call a ATF function
 - S-YIELD to willingly leave SWd and let ATF/NWd run
 - reponse to SMC that came from ATF or NWd. Actually this is very similar to S-YIELD
- After SWd was left, the system does something and will eventually enter SWd again

- can simply do this exactly where SWd has been left - the state is that sel4 is in the TZ-Thread after the SMC instruction
- SWd knows which SMC was used to leave - and what to expect on entry
 - entry with ATF-Command response
 - if we left with a ATF-Command that should be the response, continue with whatever we were doing
 - otherwise something is wrong.
 - entry with YIELD
 - seems we get CPU time, so try to use it. Call S-YIELD again if we are idle
 - check where we left
 - if we left with S-YIELD ...
 - ...due to an FIQ: resume operation where we got interrupted
 - ...because we were idle
 - run some check, but it's likely we are still idle. So do S-YIELD and leave SWd
 - note that we could end up in a YIELD-Loop here but if the whole system is idle, somebody should better start the "go to sleep process"
 - ToDo: define the "go to sleep process"
 - if we left calling an ATF command
 - that is not expected. Something is wrong. Could call S-YIELD again and hope next entry is the command response